Sistemas Distribuídos

PEER-TO-PEER

Guilherme Schnekenberg Teixeira
Luís Augusto Coelho de Souza
Matheus Heimrath Barbosa
Pedro Henrique Vianna Carvalho

CONCEITO DE SISTEMAS P2P

- Todos os nós da rede são chamados de pares (peers)
- Os pares possuem papéis semelhantes
- Cada par compartilha seus próprios recursos
- Todos podem solicitar e fornecer recursos
- Não existe uma distinção fixa entre cliente e servidor
- Os papéis se alternam conforme a necessidade

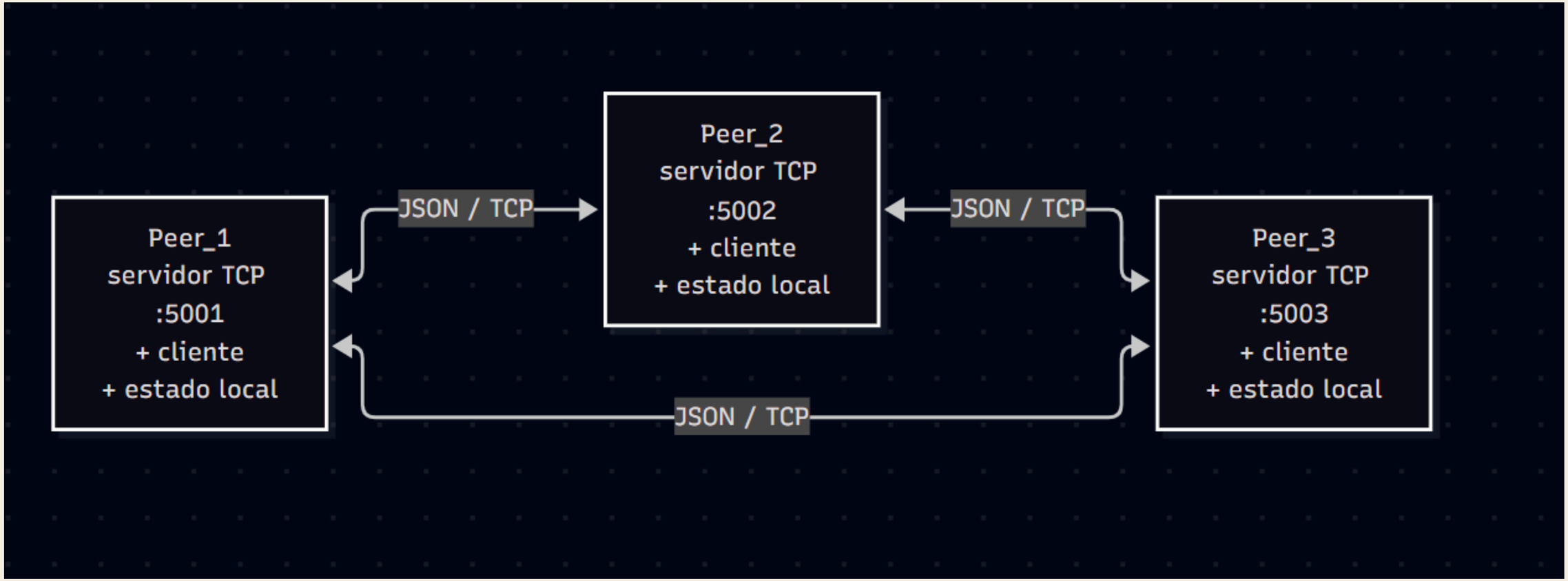
VANTAGENS E EXEMPLOS

- Distribuição do processamento e das tarefas entre os pares
- Sem ponto único de falha
- Escalabilidade: cada par novo adiciona capacidade
- Menor custo de infraestrutura central
- Exemplos reais: BitTorrent, Bitcoin/blockchain

DESVANTAGENS E FERRAMENTAS

- Consistência: cada peer tem sua própria visão do estado
- Descoberta de peers e coordenação entre eles
- Segurança e confiança em nós desconhecidos
- Sem ordem global dos eventos
- Ferramentas: sockets (Python socket/asyncio, a que a POC usa), libp2p

ARQUITETURA DA POC



DEMONSTRAÇÃO

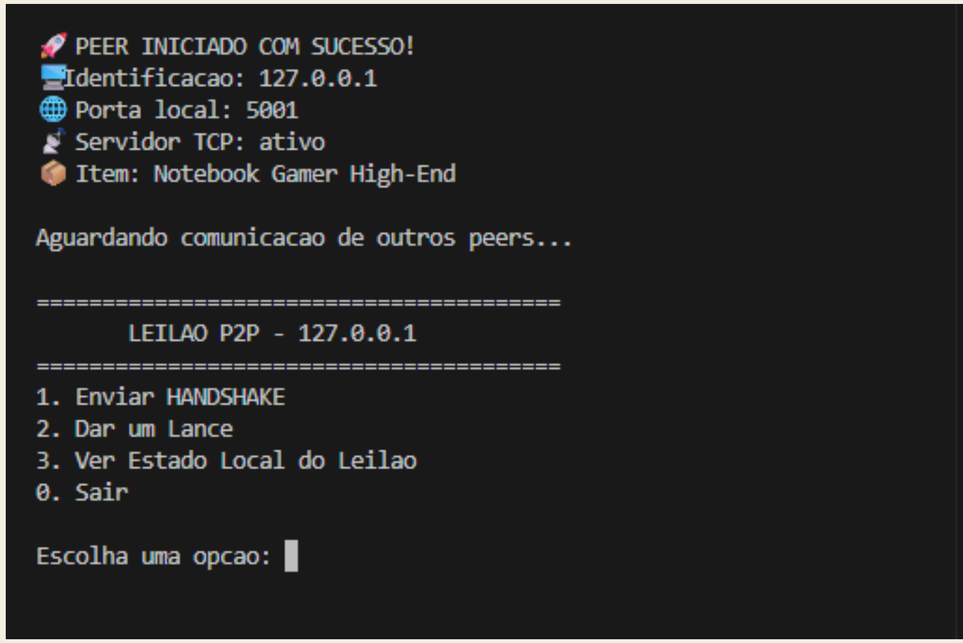
Rodar o código

python NOME_CODIGO.py 'IP_PEER' 'PORTA'

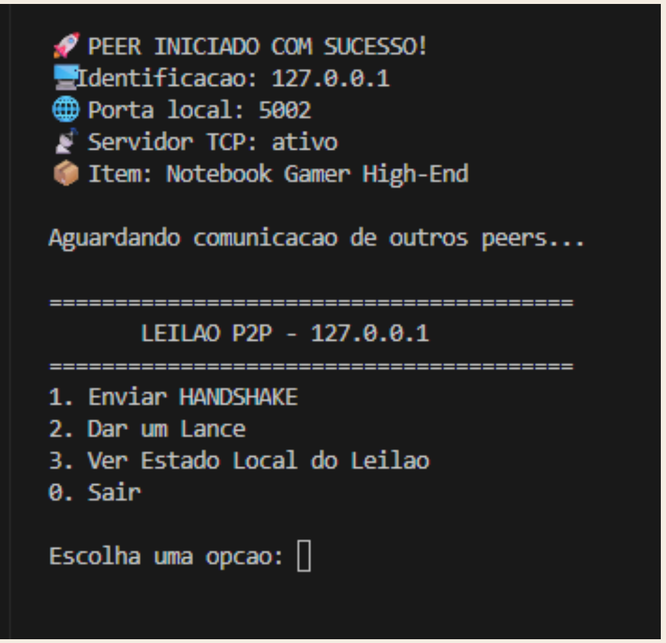
Exemplo:

python codigo-p2p.py 127.0.0.1 5001

Peer 1 (Porta 5001)



Peer 2 (Porta 5002)



DEMONSTRAÇÃO

Opção do menu: 1. Teste de conexão

Peer 1 → Enviado para Peer 2

```
========================================
       LEILAO P2P - 127.0.0.1
========================================
1. Enviar HANDSHAKE
2. Dar um Lance
3. Ver Estado Local do Leilao
0. Sair

Escolha uma opcao: 1
IP do Peer destino: 127.0.0.1
Porta do Peer destino: 5002

📤 [RESPOSTA de 127.0.0.1:5002]: {'tipo': 'HANDSHAKE_OK',
'status': 'CONECTADO', 'peer_id': '127.0.0.1', 'estado': {
'item': 'Notebook Gamer High-End', 'lance_minimo': 1000.0,
 'maior_lance': 1000.0, 'vencedor_id': None}}
```

Resposta imprime na tela do Peer 2

```
========================================
       LEILAO P2P - 127.0.0.1
========================================
1. Enviar HANDSHAKE
2. Dar um Lance
3. Ver Estado Local do Leilao
0. Sair

Escolha uma opcao:
📥 [RECEBIDO de 127.0.0.1]: HANDSHAKE -> {'tipo': 'HANDSHAKE',
'par_origem_id': '127.0.0.1', 'estado': {'item': 'Notebook Game
r High-End', 'lance_minimo': 1000.0, 'maior_lance': 1000.0, 've
ncedor_id': None}}
```

DEMONSTRAÇÃO

Opção do menu: 2. Realizar um lance

Peer 2 realiza um lance

```
Escolha uma opcao: 2
IP do Peer destino: 127.0.0.1
Porta do Peer destino: 5001
Valor do lance (R$): 2000

📤 [RESPOSTA de 127.0.0.1:5001]: {'tipo': 'LANCE_RESPOSTA',
'status': 'ACEITO', 'estado': {'item': 'Notebook Gamer High-
End', 'lance_minimo': 1000.0, 'maior_lance': 2000.0, 'venced
or_id': '127.0.0.1'}}
```

Resposta recebida no Peer 1

```
📩 [RECEBIDO de 127.0.0.1]: LANCE_PROPOSTA -> {'tipo': 'LANCE_PROPOSTA',
'par_origem_id': '127.0.0.1', 'valor': 2000.0}
✅ Lance de R$ 2000.00 ACEITO! Novo maior lance.
```

DEMONSTRAÇÃO

Opção do menu: 2. Realizar um lance (Recusado)

Tentativa de lance do Peer 1 com valor abaixo do atual

```
Escolha uma opcao: 2
IP do Peer destino: 127.0.0.1
Porta do Peer destino: 5002
Valor do lance (R$): 1500

📩 [RESPOSTA de 127.0.0.1:5002]: {'tipo': 'LANCE_RESPOSTA', 'status': 'RECUSADO',
'estado': {'item': 'Notebook Gamer High-End', 'lance_minimo': 1000.0, 'maior_lance
': 2000.0, 'vencedor_id': '127.0.0.1'}}
```

Resposta recebida no Peer 2

```
📩 [RECEBIDO de 127.0.0.1]: LANCE_PROPOSTA -> {'tipo': 'LANCE_PROPOSTA', 'par_origem_id':
 '127.0.0.1', 'valor': 1500.0}
❌ Opcao invalida. Tente novamente.ior lance atual: R$ 2000.00
```

DEMONSTRAÇÃO

Opção do menu: 3. Status atual do Leilão

```
========================================
       LEILAO P2P - 127.0.0.1
========================================
1. Enviar HANDSHAKE
2. Dar um Lance
3. Ver Estado Local do Leilao
0. Sair

Escolha uma opcao: 3

📊 ESTADO LOCAL DO LEILAO
----------------------------------
Item: Notebook Gamer High-End
Lance minimo: R$ 1000.00
Maior lance: R$ 1000.00
Vencedor: Nenhum
----------------------------------
```

EXPERIMENTO RE2 - FALHA DE UM PEER

- **Experimento:** Derrubamos um par com Ctrl+C durante o leilão, demos um lance em outro par e reiniciamos o par derrubado;

- **Resultado:** os outros pares continuaram funcionando. Quem deu o lance viu "*[AVISO] Peer_X inacessível*" e o resumo "*1 aceitaram*, *0 recusaram*, *1 inacessível*". Ao voltar, o par recuperou pelo handshake o maior lance e o horário de encerramento;

- **Por quê:** não há ponto único de falha, e o estado está replicado em todos os pares;

- **Limitação:** se todos os peers caírem, o estado se perde, porque não há persistência em disco.

EXPERIMENTOS RE3 - LANCES SIMULTÂNEOS

- **Experimento:** dois pares deram R$1500 ao mesmo tempo. Fizemos uma rodada com a regra de desempate e outra com *--sem-desempate*;
- **Com desempate:** os pares receberam os lances em ordens diferentes, mas todos aplicaram a mesma regra (menor Lamport). Resultado: *CONSENSO*;
- **Sem desempate:** cada par ficou com o lance que chegou primeiro nele. Resultado: *DIVERGÊNCIA* (Peer_2 discordou);
- **Por quê:** em P2P não há exclusão mútua entre pares. A convergência só vem de uma regra determinística igual para todos

USO DA I.A.

- **Prompt 1:** explique a arquitetura P2P (peer-to-peer) em comparação com cliente/servidor: como funciona, vantagens, desvantagens e exemplos reais de uso em grandes empresas
  - **Aproveitado:** entendimento da arquitetura, base de vantagens e desvantagens

- **Prompt 2:** proponha um formato de mensagens JSON para um leilão P2P em que cada par envia seus lances diretamente aos outros. preciso de mensagens para entrar na rede, dar lance, responder aceito/recusado e anunciar o vencedor no encerramento
  - **Aproveitado:** tipos `HANDSHAKE`/`HANDSHAKE_OK`, `LANCE_PROPOSTA`/`LANCE_RESPOSTA` e `VENCEDOR_ANUNCIO`, com uma linha JSON por mensagem (`rede.py`)
  - **Adaptado:** campos `porta` (para o remetente ser registrado como par) e `lamport` (para ordenar os eventos)

USO DA I.A. - CASO DE ERRO

- **Contexto:** um par enviava HANDSHAKE e o outro par só processava a mensagem depois de escolher uma opção no menu dele. Até lá, a mensagem ficava parada. Do mesmo modo, quando o outro par escolhia enviar um HANDSHAKE como a ação que quebrasse esse estado congelado, entrava-se em um loop, pois agora o outro par precisa escolher uma ação do menu antes de receber o HANDSHAKE;

- **IA**: disse que esse comportamento era esperado do P2P;

- **Como percebemos:** o comportamento contraria a ideia de P2P, em que cada par é servidor o tempo todo e deve responder aos outros a qualquer momento, sem depender do usuário. Também contraria o RF4

- **Correção:** a leitura do teclado passou para uma thread separada (classe `LeitorTeclado` em `app.py`), que entrega as linhas ao event loop por uma fila. Assim, o servidor continua atendendo mensagens enquanto o menu espera.

CONCLUSÃO

- **Quando escolher P2P?**
  - Não há uma autoridade central confiável;
  - Disponibilidade e tolerância a falhas importam mais que consistência imediata.

- **Quando não escolher? (o caso do leilão real)**
  - É preciso um vencedor oficial e indiscutível;
  - Há dinheiro envolvido: confiança, autenticação e auditoria.

- **Problemas distribuídos que apareceram**
  - Ordem dos lances: sem relógio global, resolvemos com Lamport + desempate determinístico;
  - Horário de encerramento: “10h00 pelo relógio de qual máquina?”;
  - Consenso sobre o vencedor: detectamos a divergência, mas corrigir exigiria Paxos ou Raft.

Sistemas Distribuídos

PEER-TO-PEER

Guilherme Schnekenberg Teixeira
Luís Augusto Coelho de Souza
Matheus Heimrath Barbosa
Pedro Henrique Vianna Carvalho